Data-Driven Computer Animation

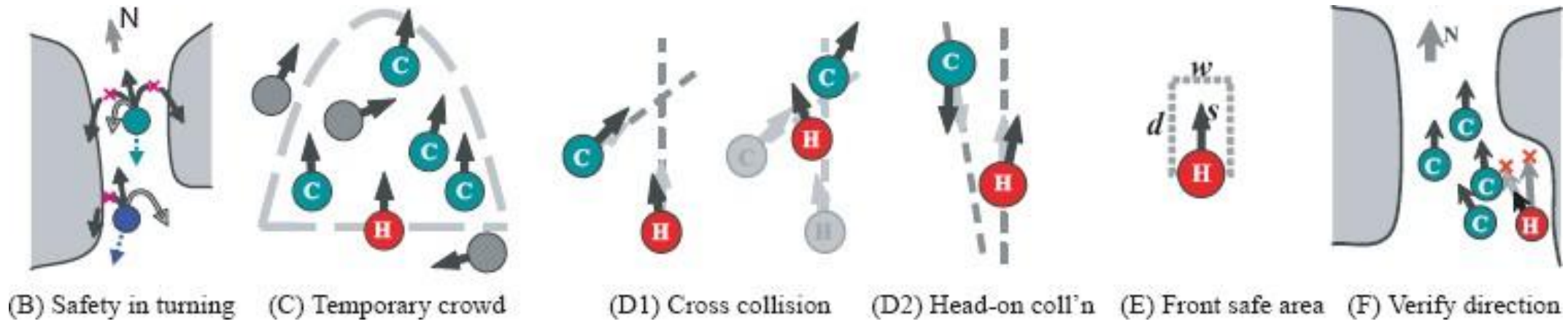
Crowd Simulation by Machine Learning

Taku Komura

Agent-based character control

- Flocking, autonomous pedestrians, etc.
  - Requires tuning parameters to observe desired behavior
  - Combining rules and physics for animating the agents

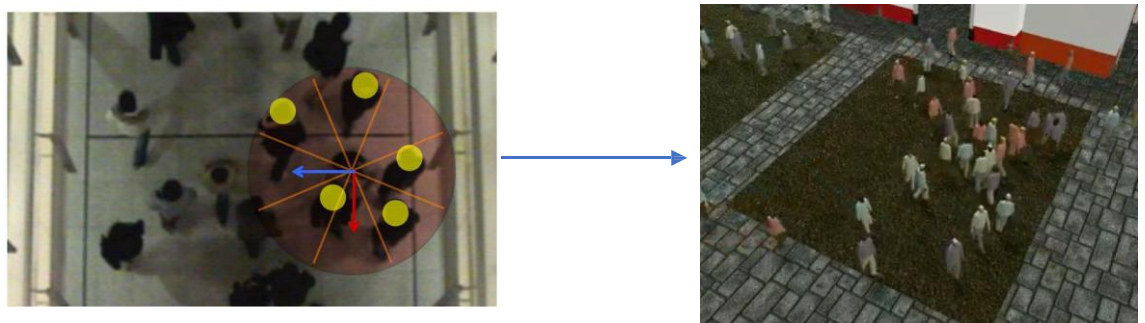
Can we do crowd simulation without tuning such parameters?



(B) Safety in turning (C) Temporary crowd (D1) Cross collision (D2) Head-on coll'n (E) Front safe area (F) Verify direction

Crowd Simulation by Machine Learning

- There can be two approaches.
- **Data-driven techniques (supervised learning)**
  - **Observe the crowd behavior in videos etc.**
  - **Learn how people behaves and responds in every state**
- Reinforcement Learning
  - You define a reward function for all the agents, and then let them explore, and learn to find the best action at each state
  - The trained agents select the best action at each state



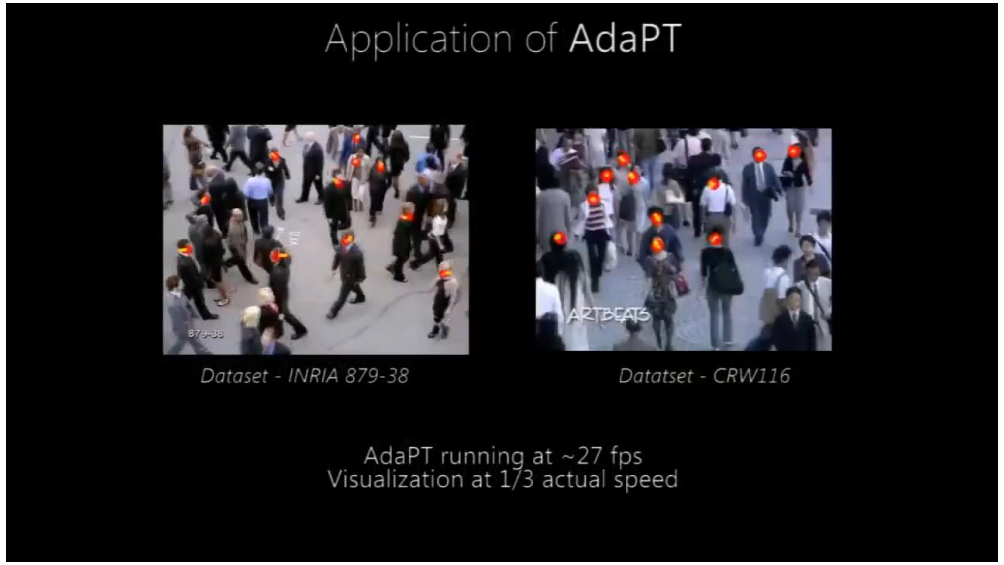
How to learn the data?

- Need to first capture data
- One approach: Use the videos from the top and track people's movements
- We can obtain the state-action pairs how humans behave in each situation



Motion tracking in the videos

- The trajectory of the pedestrians in the videos can be tracked using computer vision techniques
- We can use the collected data for training the model





State-Action Pair

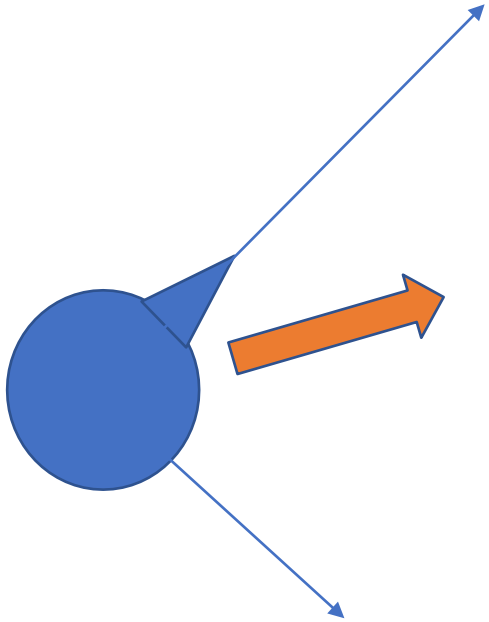
- State **s**: the velocity of the agent, the formation of the nearby agents, the agent's intention

- Action **a**: a two-dimensional vector that corresponds to acceleration – or can be the pair of current velocity and next velocity

- We wish to collect such pair data (**s, a**) so that we can train a system that can predict the action given the state

State Vector : Velocity

- The agent’s velocity is the first state vector
- The velocity must be defined with respect to the person – in the local coordinate defined at the person’s body

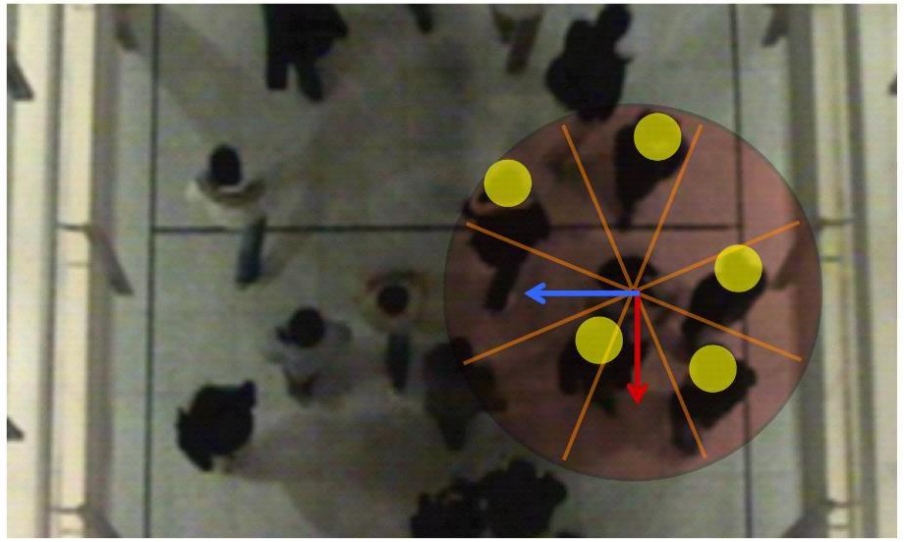


State Vector (2): Neighborhood Formation

- Divide the circle around the agent into eight radial regions
- We set a threshold value to r - only consider the other agents within the distance r
- The distance to the other agent within each region (d) is used to compute the feature value:

$$f = \begin{cases} (r-d)^2/r^2, & \text{if } d < r \\ 0, & \text{otherwise} \end{cases}$$

- To consider the temporal information, compute this for two adjacent frames



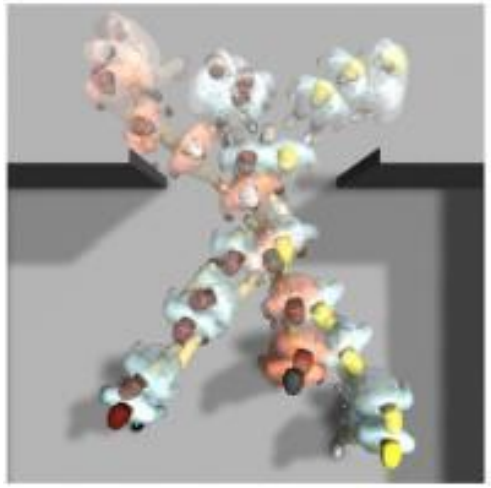
State Vector (3): Intention

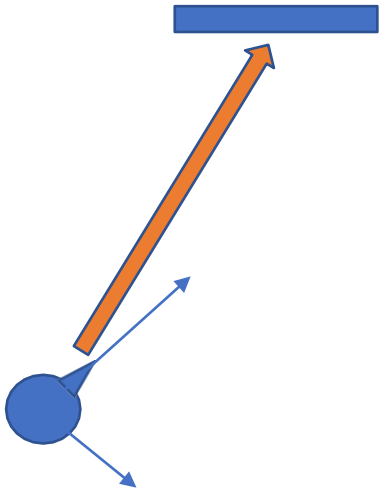
- The person usually has an intention to walk from one place to another
- We can pick 10 frames of the motion in the past and the future, and then average the motion to compute the person's high-level goal.



State Vector (4): Pivots

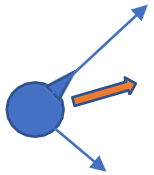
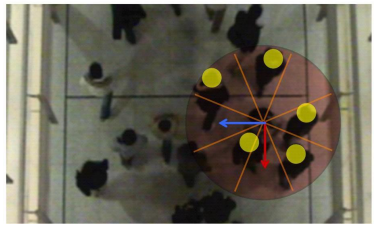
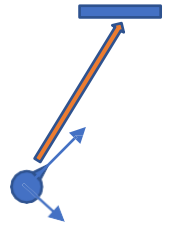
- There can be objects in the environment, such as obstacles or doors.
- They play important roles in the movements and their position with respect to the body must be considered
- Its position with respect to the body is added as additional information



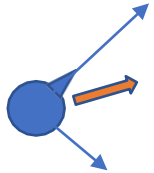


State-Action Pair

- State **s**: the velocity of the agent, the formation of the nearby agents, the agent's intention

s = (   )

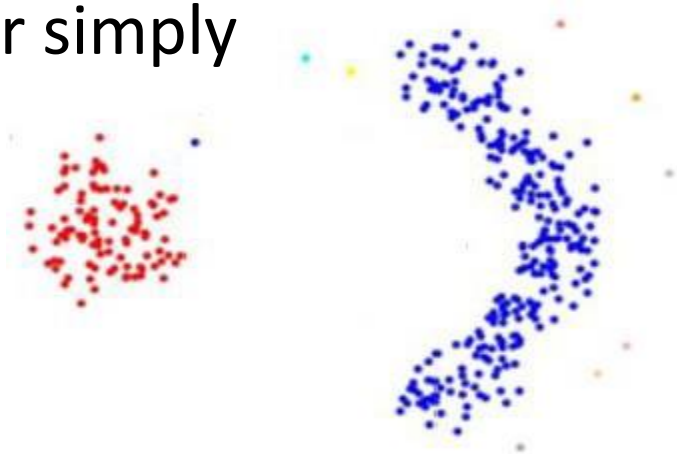
- Action **a**: a two-dimensional vector that corresponds a two-dimensional vector that corresponds to acceleration – or can be the pair of current velocity and next velocity

a = ()

- We wish to collect such pair data (**s, a**) so that we can train a system that can predict the action given the state

Prediction by Nearest Neighbors

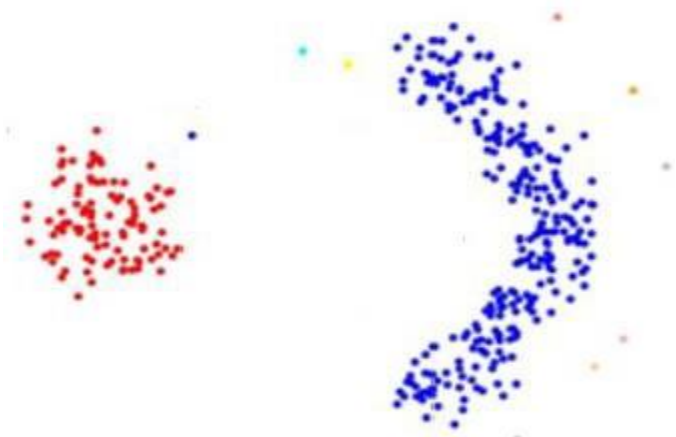
- Given the current state s, we want to select the action.
- We first search in the database for the k-nearest neighbors s_i $(1 \leq i \leq k)$ similar to the current state s
- Each state $s_i$has the corresponding a_i
- As the human motion is not deterministic, the actions could be very different
- For example, when walking, you may start to run when you notice you have less time, stop and start to talk to somebody, or simply continue to walk.
- Thus, we cluster the actions



Clusters of actions

Probabilistic Model

- Each cluster could be corresponding to different type of actions
- Which cluster to select from the action is the next issue
- We select the cluster in a probabilistic manner



Clusters of actions

Probabilistic Model (2)

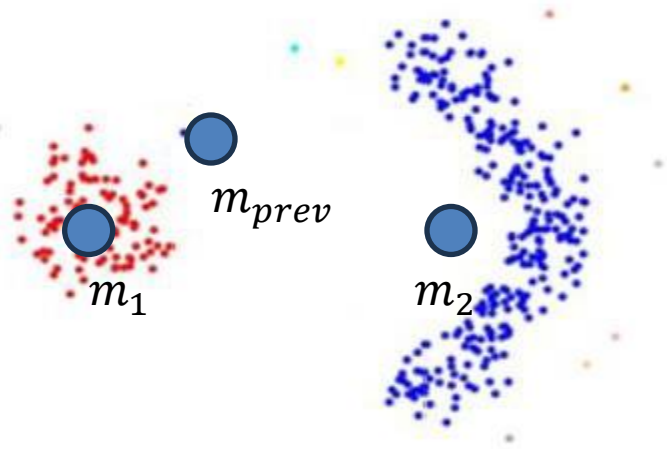
- The probability of the i-th cluster to be selected is computed such that a new cluster that is close to the average of the previously selected cluster.

  (so that the action does not switch so often)

- The probability that the i-th cluster to be selected is inverse proportional to the distance between the mean of the i-th cluster ($\mathbf{m}_i$) and the mean of the previously selected cluster ($\mathbf{m}_{prev}$)

$$p_i = \frac{1/\|\mathbf{m}_i - \mathbf{m}_{prev}\|}{\sum_{j=1}^{K} 1/\|\mathbf{m}_j - \mathbf{m}_{prev}\|}$$

where *K* is the number of clusters



Locally Weighted Linear Regression

- Using the samples in the cluster $(\mathbf{s_i}, \mathbf{a_i})$, a linear regression model is constructed that predicts the action $\mathbf{a}$ given the state $\mathbf{s}$

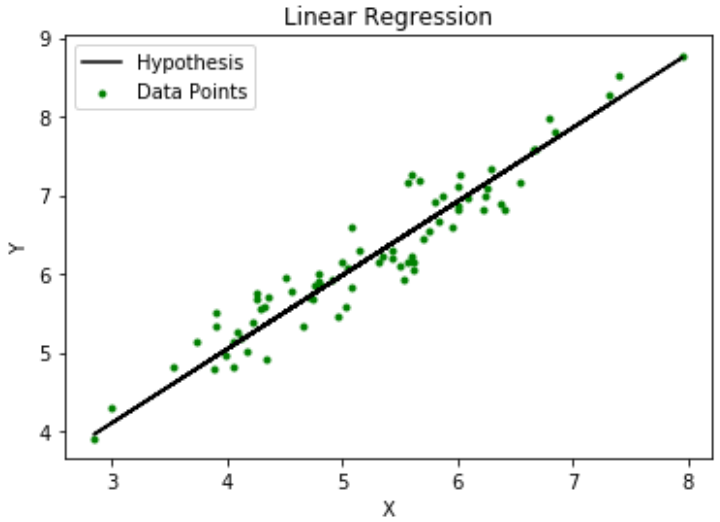
$$\mathbf{a} = \mathbf{s}\,\boldsymbol{\beta}$$

- Locally weighted linear regression adjusts the coefficient vector according to the sample $\mathbf{s}$

Linear Regression

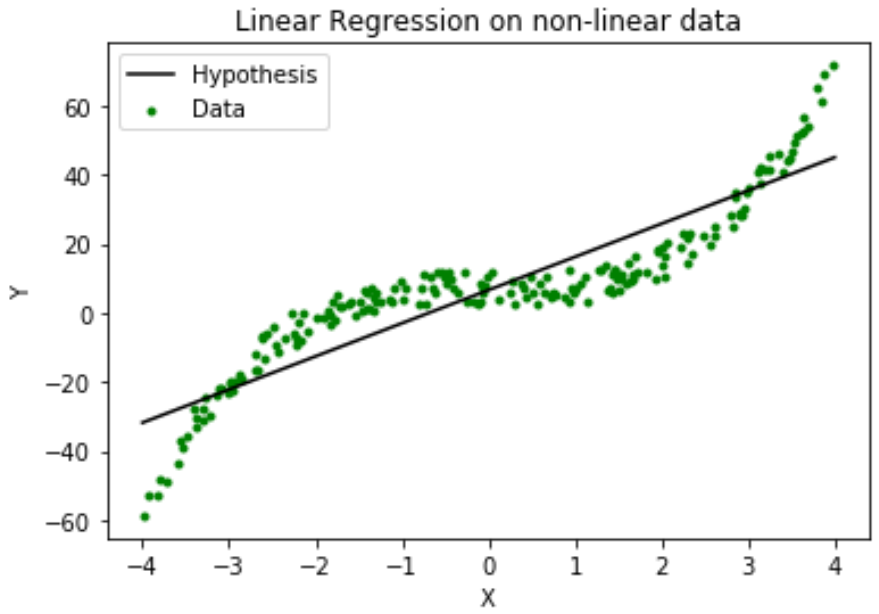
In linear regression, a linear model is constructed to predict the output from the input

This is done by minimizing the cost $J = \sum_i^k \|\boldsymbol{s}_i\boldsymbol{\beta} - \boldsymbol{a}_i\|^2$



Linear Regression (2)

But this doesn't work very well when there is a nonlinear relationship between the input and the output



Locally Weighted Linear Regression

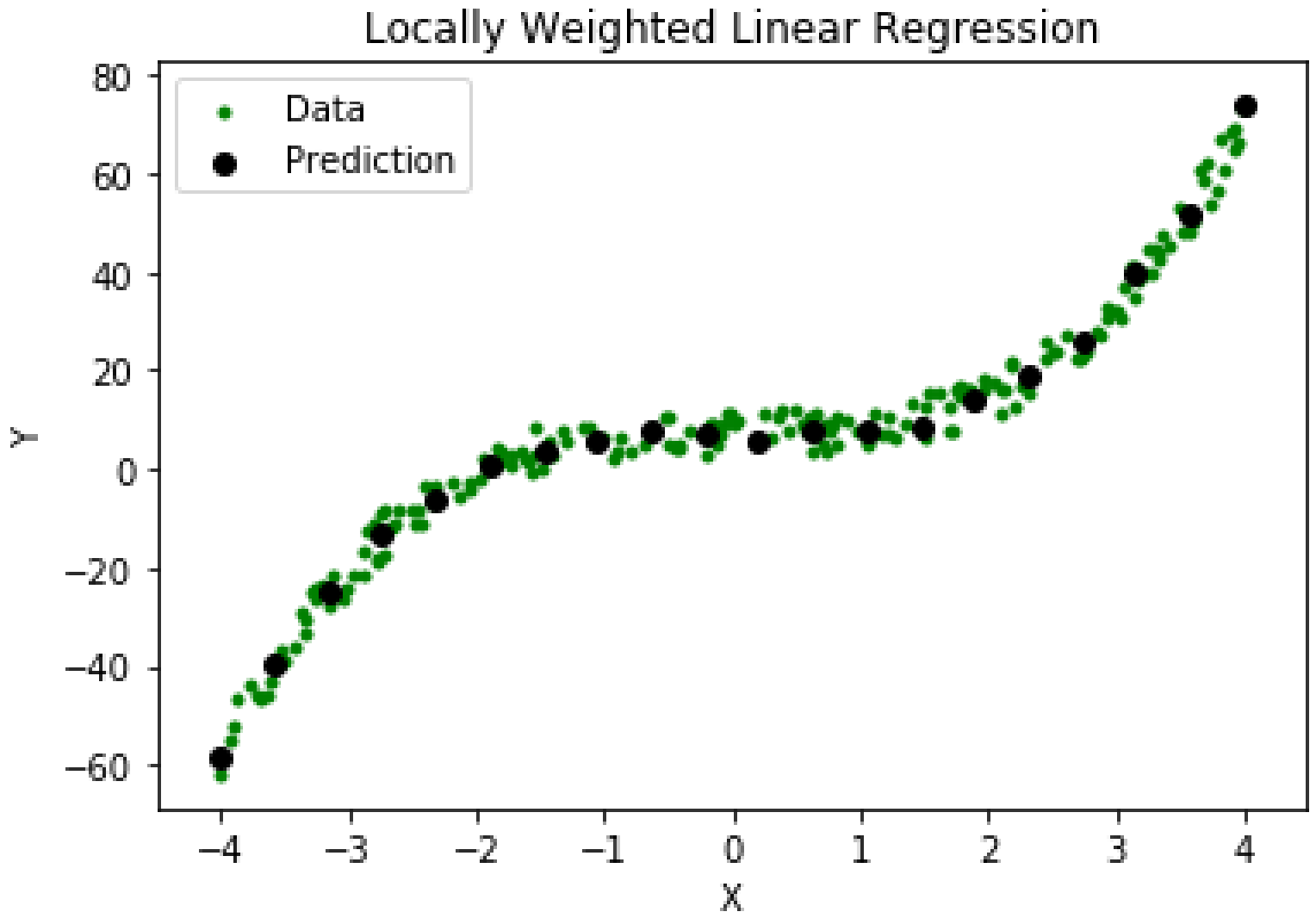
Instead of using a fixed $\boldsymbol{\beta}$ for all the query points, it is recomputed according to the given query point

Here we minimize the following equation instead:

$$J = \sum_{i}^{k} w_i \|\boldsymbol{s}_i\boldsymbol{\beta} - \boldsymbol{\alpha}_i\|^2$$

where w_i is a weighting, whose value decreases as the sample gets farther from the query point $\boldsymbol{s}$

We can use $w_i = \exp\left(\frac{-\|\boldsymbol{s}_i - \boldsymbol{s}\|^2}{\sigma}\right)$


Locally Weighted Linear Regression
Data
Prediction
80
60
40
20
0
−20
−40
−60
Y
−4
−3
−2
−1
0
1
2
3
4
X

Locally Weighted Linear Regression

- Given the state *s*, the action *a* is computed by

$$\mathbf{a}=\mathbf{s}\,\boldsymbol{\beta}$$

where $\boldsymbol{\beta}$ is computed by

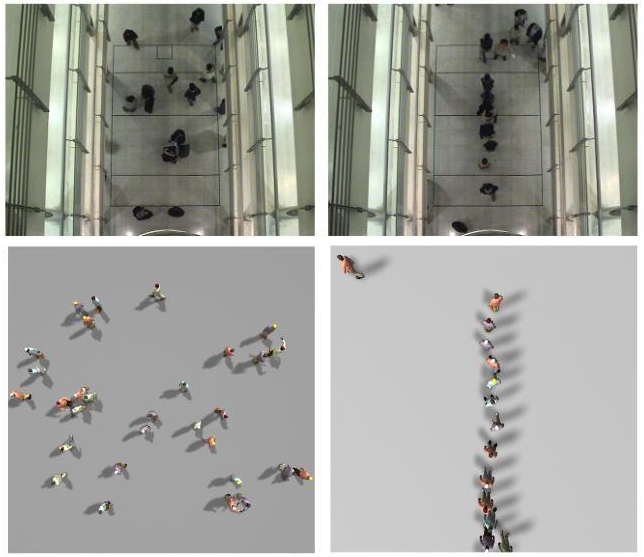
$$\beta = (S^T W S)^{-1} S^T W \mathbf{x},$$

Where *S* is a matrix that is composed with all $\boldsymbol{s}_i$ and **x** is a matrix that is composed of all $\boldsymbol{a}_i$

and *W* is a diagonal matrix with $W_i = \exp\left(\frac{-\|\boldsymbol{s}_i - \boldsymbol{s}\|^2}{\sigma}\right)$

High-Level Behavior

- The model proposed is good to produce low level response motion like avoiding others but not for high level behaviors, such as joining a small group of agents for interaction and then leaving.
- We can produce a finite-state-machine, that determines the series of low-level actions

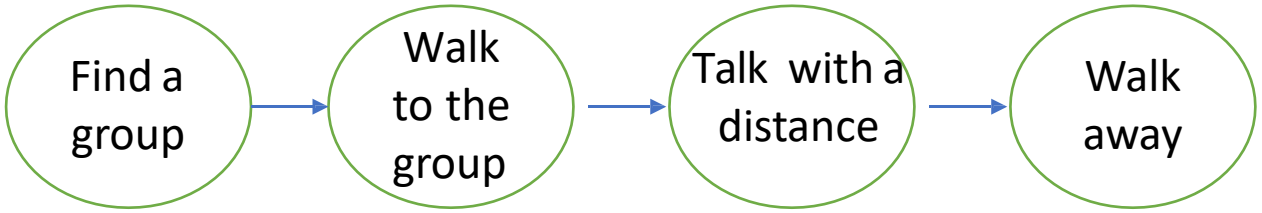


High-Level Behavior (2)

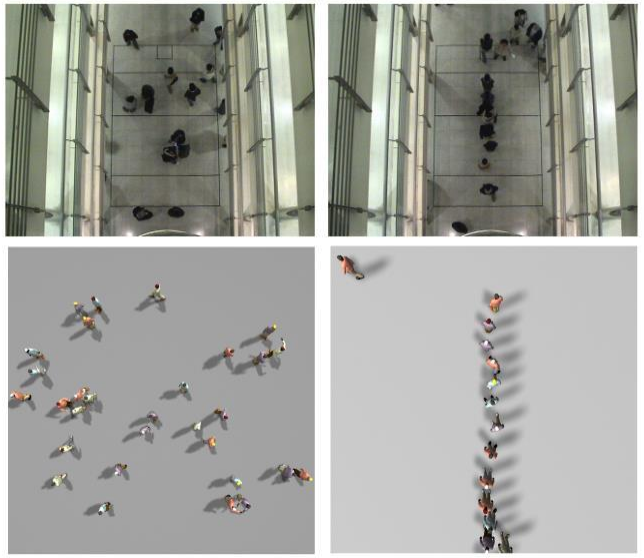
- For example:

Chatting: find a group nearby, walk towards the group, keep a distance with the others and stay there for 1 minute, and then leave.

Queuing: find the end of a queue, walk to the end of the queue, keep a distance of one meter while idling, move forward

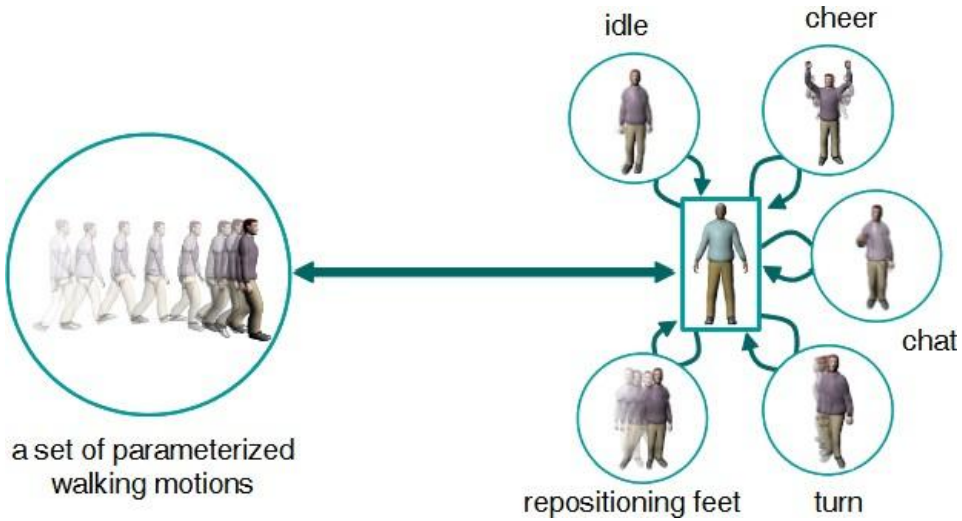


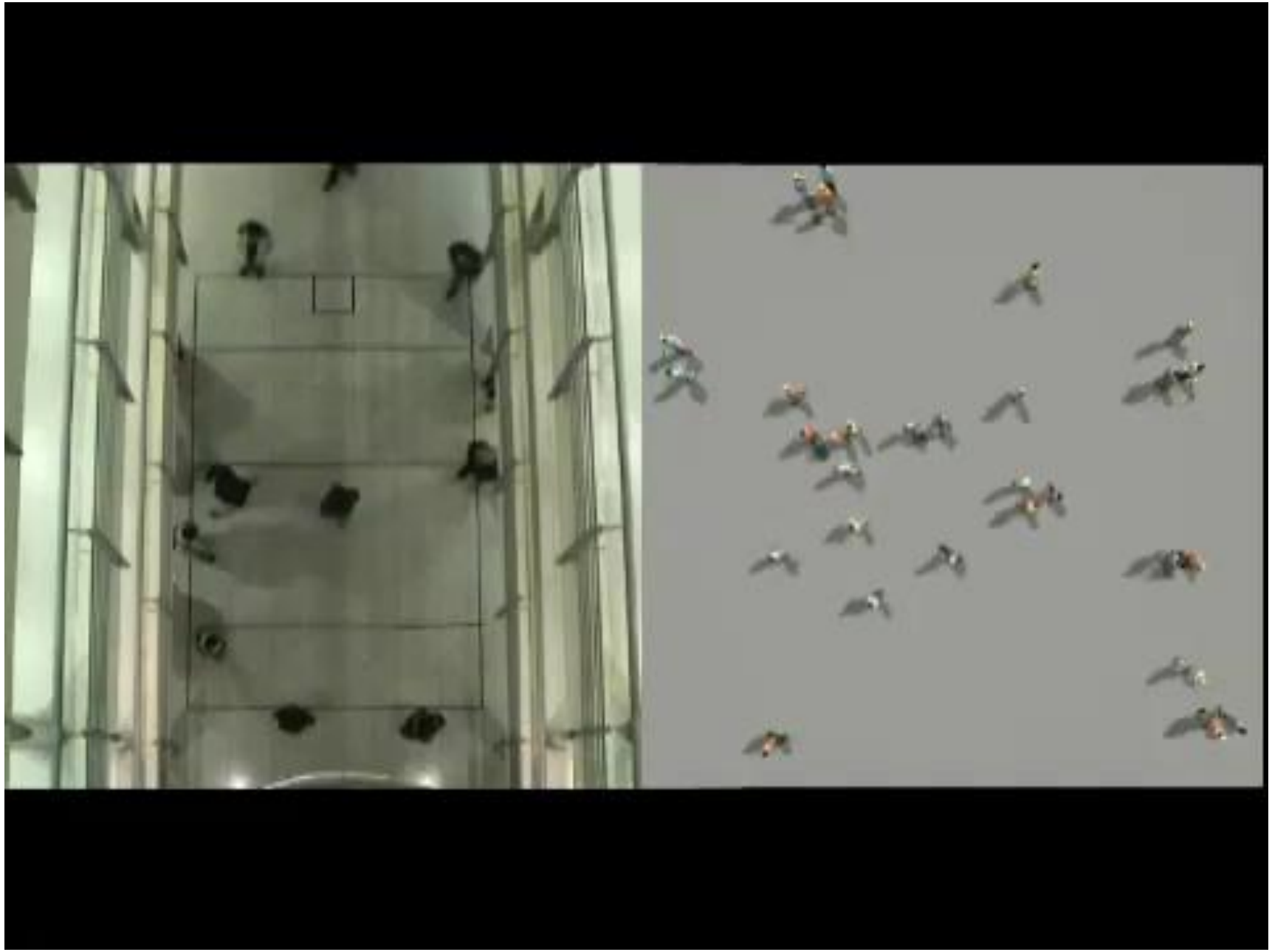
Finite State Machine for Chatting

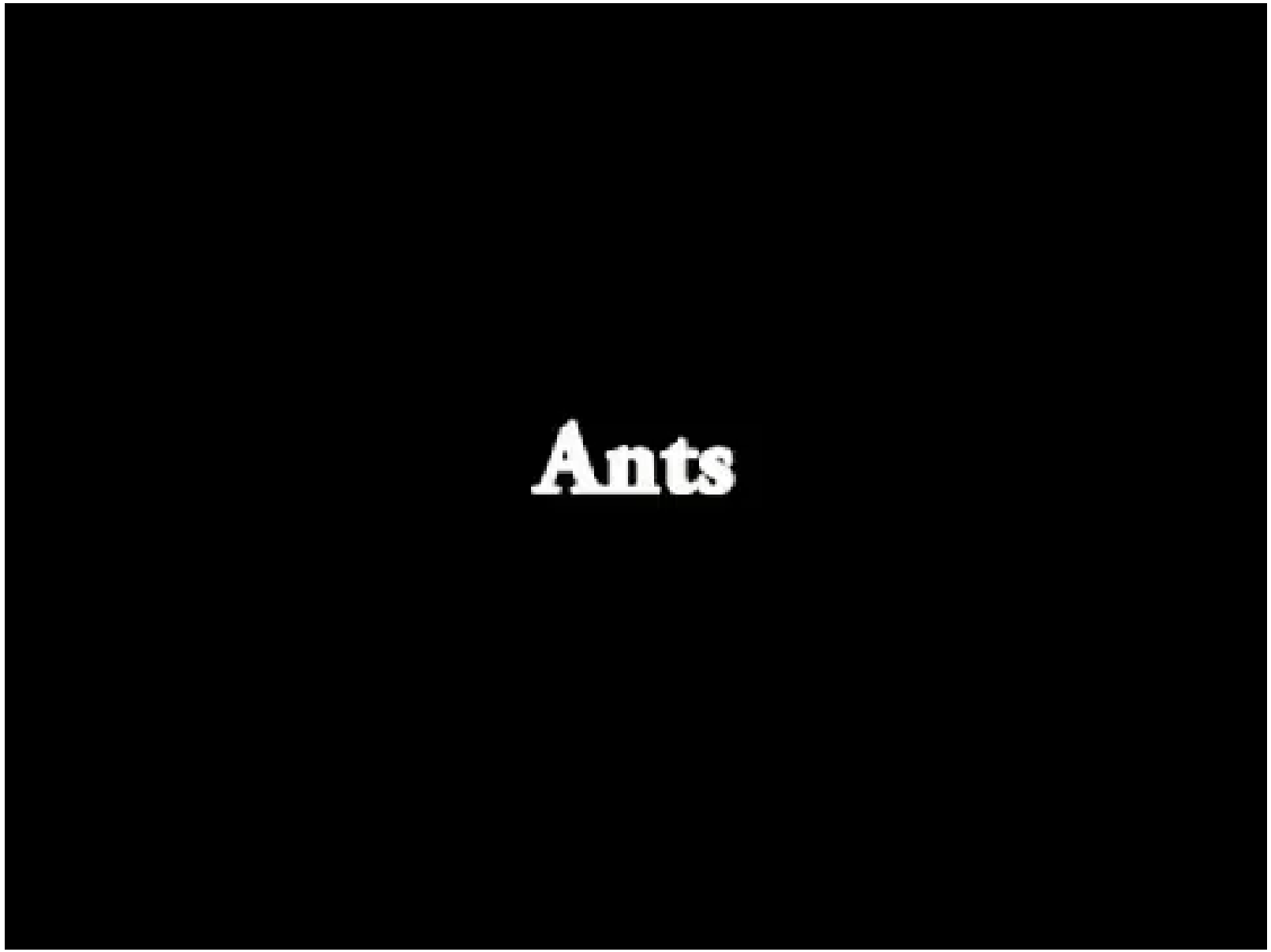


Full Body Motion

- A motion graph for full body motion is produced using the motion capture data.
- Motions to follow a trajectory is produced by blending walking/running motions in different curvatures

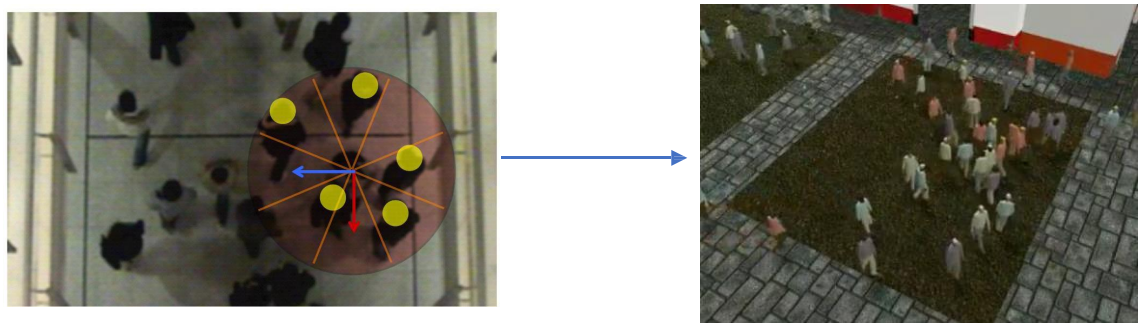





Ants

Crowd Simulation by Machine Learning

- There can be two approaches.
- Data-driven techniques (supervised learning)
  - Observe the crowd behavior in videos etc.
  - Learn how people behaves and responds in every state
- **Reinforcement Learning**
  - **You define a reward function for all the agents, and then let them explore, and learn to find the best action at each state**
  - **The trained agents select the best action at each state**



Reinforcement Learning

- Reinforcement learning (RL) solves a sequential decision-making problem

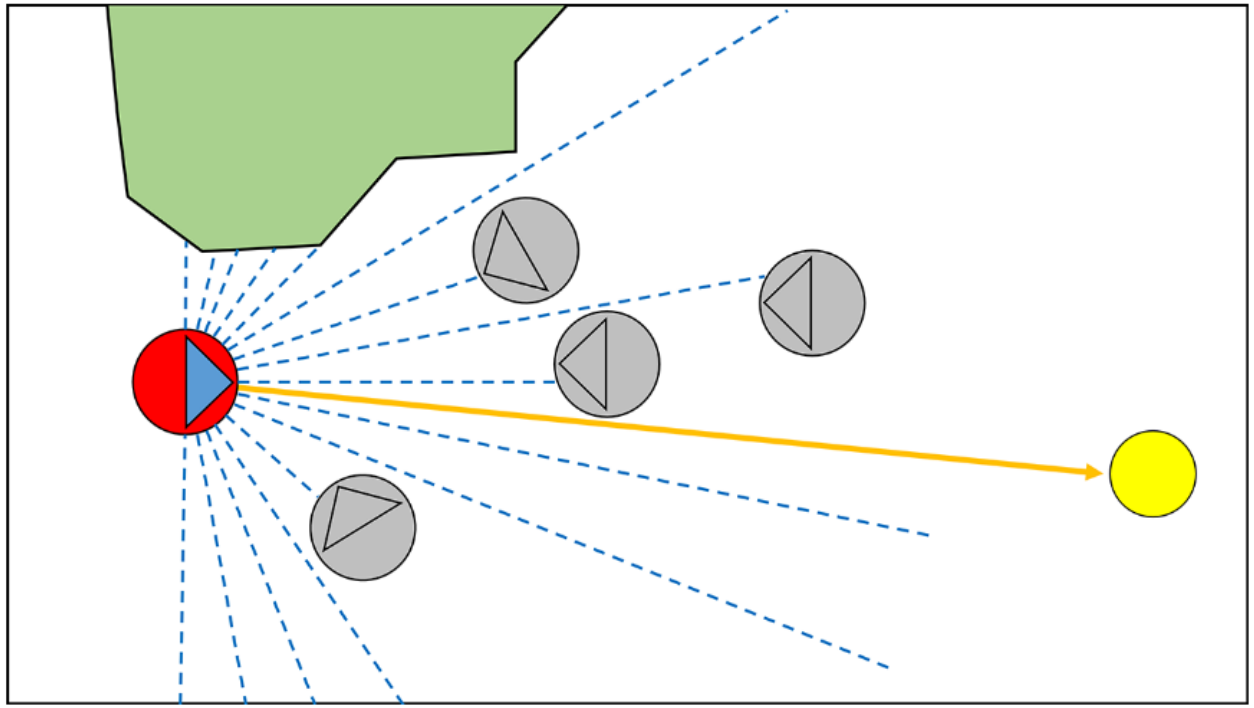
- Given the current state **s**, a policy is trained such that it selects an action **a** such that the accumulated reward defined as follows is maximized:

$$\eta(\pi) = E_{s_0, a_0, \cdots}\left[\sum_{t=0}^{\infty} \gamma^t r_t\right]$$

where r is a reward obtained and γ is a discount factor which prevents the sum of rewards from becoming infinity

State Representation

- The state s of an agent is composed of
  - the relative position of the agent with respect to the goal, and
  - the time series of depth maps from the previous K frames



Reward for character control in crowd animation

- We wish the character to reach the goal, avoid collisions, and produce smooth motion

$$\mathcal{R} = r_{goal} + r_{collision} + r_{smooth}$$

- The first term goal term penalizes a large deviation of the agent from the goal

$$r_{goal} = w_1(\bar{l} - l)$$

where $\bar{l}$ and l are distances to the goal from the agent at previous and current time-steps, respectively

Reward for character control in crowd animation (2)

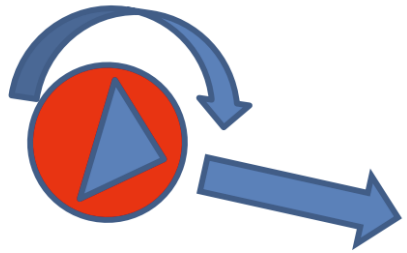
- The collision term checks agent-agent or agent-obstacle collisions, then gives a penalty if it is occurred

$$r_{collision} = \begin{cases} -w_2 & \text{if collision occurs} \\ 0 & \text{otherwise} \end{cases}$$

Reward for character control in crowd animation (3)

The smoothness reward r_{smooth} encourages the character to move smoothly,

- penalizing to make sudden changes in velocity
- penalizing the velocity to be too large



Reinforcement Learning

- Reinforcement learning (RL) solves a sequential decision-making problem

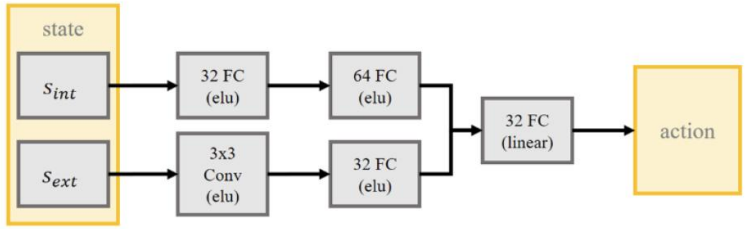
- Given the current state **s**, a policy is trained such that it selects an action **a** such that the accumulated reward defined as follows is maximized:

$$\eta(\pi) = E_{s_0, a_0, \cdots}\left[\sum_{t=0}^{\infty} \gamma^t r_t\right]$$

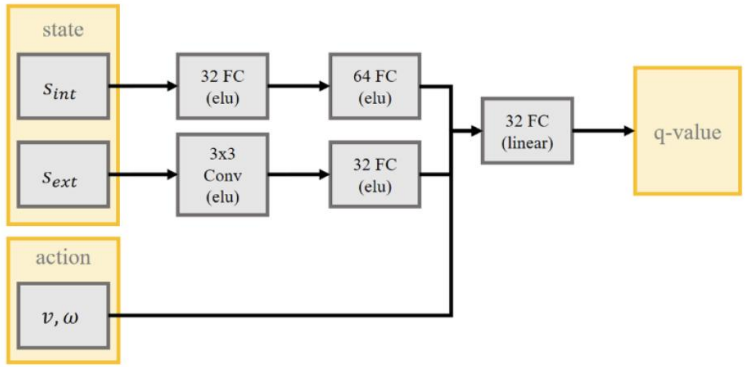
where r is a reward obtained and γ is a discount factor which prevents the sum of rewards from becoming infinity

Learning the Policy

- We can utilize deep reinforcement learning methods where the policy is learned by neural networks.
- More scalable and can generalize the training data



$A(s)$



$V(s)$ or $Q(s, a)$

Training the Critic (Value Function)

$V\ (s)$ (or $Q(s,a)$) is trained by MSE loss
Compute $V\ (s)$
Select an action by the Actor A(s) and then find the next state s'
Compute the new value for s: $\mathrm{V}' = r + \gamma V\ (s')$
Train the critic with MSE: $\|\mathrm{V}(s) - \mathrm{V}'\|$

Training the Actor

- Using $V\ (s)$ compute the value at the current state
- Select an action by the Actor A(s) and then find the next state s'
- Compute the value at the next state for s':
- $\mathrm{V}' = r + \gamma V\ (s')$
- Compute the advantage: A(s,a)= $r + \gamma V\ (s') - V(s)$
- Use the A(s,a) to update Actor

Crowd Simulation by Deep Reinforcement Learning

Jaedong Lee, Jungdam Won, Jehee Lee
Seoul National University

Summary

Supervised learning learns from the data

- Can produce not only avoidance but chatting, interacting, etc.
- Requires multiple stages of modelling

Deep reinforcement learning doesn’t require training data:

- Learns a policy that maximizes the reward
- It cannot produce motions such as chatting, relaxing, etc., unless the reward is well defined

References

- Lee et al. “Group Behavior from Video: A Data-Driven Approach to Crowd Simulation”, SCA 2007
- Lee et al. “Crowd simulation by deep reinforcement learning”, MIG 2018
- Aniket Bera, Nico Galoppo, Dillon Sharlet, Adam Lake, Dinesh Manocha, “AdaPT: Real-time Adaptive Pedestrian Tracking for crowded scenes”, ICRA 2014
- Locally weighted Linear Regression: https://www.geeksforgeeks.org/ml-locally-weighted-linear-regression/